

# Pensée informatique

Écrire un programme, c'est décrire une méthode assez précisément pour qu'une machine l'exécute sans rien deviner. C'est la même exigence que celle d'une démonstration.

|                                                                                                                               |                                                                       |                                  |                                              |
|-------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|----------------------------------|----------------------------------------------|
| <b>PRIORITÉ</b><br>●○○ <b>COMPLÉMENT</b><br>Peu de points aux évaluations, mais la rigueur acquise ici sert partout ailleurs. | <b>NIVEAU</b><br><b>6<sup>e</sup> · 5<sup>e</sup> · 4<sup>e</sup></b> | <b>RÉVISION</b><br><b>15 min</b> | <b>DOMAINE</b><br><b>Pensée informatique</b> |
|-------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|----------------------------------|----------------------------------------------|

## AUTOMATISMES — à restituer immédiatement, sans poser de calcul

- Distinguer une instruction d'une séquence d'instructions **6<sup>e</sup>**
- Exécuter à la main une séquence d'instructions **6<sup>e</sup>**
- Repérer les entrées et les sorties d'un programme
- Prévoir la valeur d'une expression avant de l'exécuter **5<sup>e</sup>**
- Une boucle « répéter  $n$  fois » remplace  $n$  copies de la même séquence **RÈGLE**

6<sup>e</sup> ou 5<sup>e</sup> : niveau auquel l'attendu figure au programme. **Règle** : formulation de synthèse.

## 1. Le vocabulaire

### DÉFINITION

Une **instruction** est un ordre élémentaire : avancer de 50, tourner de 90°, ajouter 3.

Une **séquence** est une suite d'instructions exécutées **dans l'ordre**.

Une **entrée** est une donnée fournie au programme ; une **sortie** est ce qu'il produit.

### L'ORDRE CHANGE TOUT

« Avancer de 50, puis tourner de 90° » ne donne pas le même dessin que « tourner de 90°, puis avancer de 50 ». Une séquence n'est pas un sac d'instructions : c'est une suite ordonnée.

## 2. Les programmes de calcul

Un programme de calcul est déjà un algorithme. Le programme de 5<sup>e</sup> demande de savoir le traduire en **une seule expression**.

### MÉTHODE

« Choisis un nombre, ajoute 2, multiplie par 4, retire 3 »

devient  $(n + 2) \times 4 - 3$ .

Les parenthèses traduisent l'ordre des instructions. Sans elles,  $n + 2 \times 4 - 3$  décrirait un tout autre programme.

| Nombre choisi | Après +2 | Après $\times 4$ | Après -3 |
|---------------|----------|------------------|----------|
| 1             | 3        | 12               | 9        |
| 5             | 7        | 28               | 25       |
| 10            | 12       | 48               | 45       |

Ce tableau de valeurs est exactement ce qu'on obtiendrait en exécutant le programme trois fois. Le remplir à la main **avant** de lancer la machine est un attendu explicite du programme : « prévoir la valeur d'une expression informatique avant son exécution ».

### 3. La boucle

#### BOUCLE INCONDITIONNELLE

Répéter  $n$  fois une même séquence. En 5<sup>e</sup>, seul ce type de boucle est au programme : le nombre de répétitions est connu à l'avance.

#### DESSINER UN CARRÉ

Sans boucle, il faut huit instructions :

*avancer de 100, tourner de 90°, avancer de 100, tourner de 90°...*

Avec une boucle, deux suffisent :

**répéter 4 fois** [ avancer de 100 ; tourner de 90° ]

Pour un triangle équilatéral : **répéter 3 fois** [ avancer de 100 ; tourner de 120° ].

#### L'ANGLE DE ROTATION N'EST PAS L'ANGLE DE LA FIGURE

Pour un triangle équilatéral, dont les angles mesurent 60°, on tourne de 120° à chaque sommet. Le lutin tourne de l'angle **extérieur**, et la somme de ces rotations vaut toujours 360° pour un tour complet :  $360 \div 3 = 120$ .

### 4. Les motifs évolutifs

Le programme relie l'algorithmique à la **pensée algébrique** par les suites de motifs. On cherche la régularité, puis on prédit une étape lointaine sans la dessiner.

| Étape              | 1 | 2 | 3  | 4  | ... |
|--------------------|---|---|----|----|-----|
| Nombre de carreaux | 4 | 7 | 10 | 13 | ?   |

#### TROUVER LA RÈGLE

On passe d'une étape à la suivante en ajoutant 3. À l'étape 1 il y a 4 carreaux, soit  $1 + 3 \times 1$ . À l'étape 2,  $1 + 3 \times 2 = 7$ . À l'étape  $n$  :  $1 + 3n$ .

L'étape 100 comporterait donc  $1 + 3 \times 100 = 301$  carreaux — sans rien dessiner. C'est le pas décisif vers le calcul littéral de la fiche 5.

## 5. 4<sup>e</sup> Variables et conditions

### LA VARIABLE INFORMATIQUE

Une **variable** est une case mémoire portant un nom, dont le contenu peut changer pendant l'exécution. `score <- score + 1` se lit : « prends la valeur actuelle de `score`, ajoute 1, et range le résultat dans `score` ».

### LE SIGNE N'A PAS LE MÊME SENS QU'EN MATHÉMATIQUES

En mathématiques,  $x = x + 1$  est une égalité impossible. En informatique, `x <- x + 1` est une **instruction** parfaitement valide : elle range dans `x` son ancienne valeur augmentée de 1.

### L'INSTRUCTION CONDITIONNELLE

`si condition alors ... sinon ...` : le programme choisit son chemin selon que la condition est vraie ou fausse.

### UN EXEMPLE COMPLET

`> si note ≥ 10 > alors afficher « admis » > sinon afficher « ajourné »`

La condition est un test qui ne peut valoir que **vrai** ou **faux** :  $\geq$ ,  $\leq$ ,  $=$ , différent de.

### CE QU'IL FAUT SAVOIR FAIRE, AU MINIMUM

- Exécuter à la main une séquence donnée et dire ce qu'elle produit.
- Repérer ce qui entre dans un programme et ce qui en sort.
- Modifier un paramètre d'un programme existant et prévoir l'effet.
- Remplacer des répétitions par une boucle.